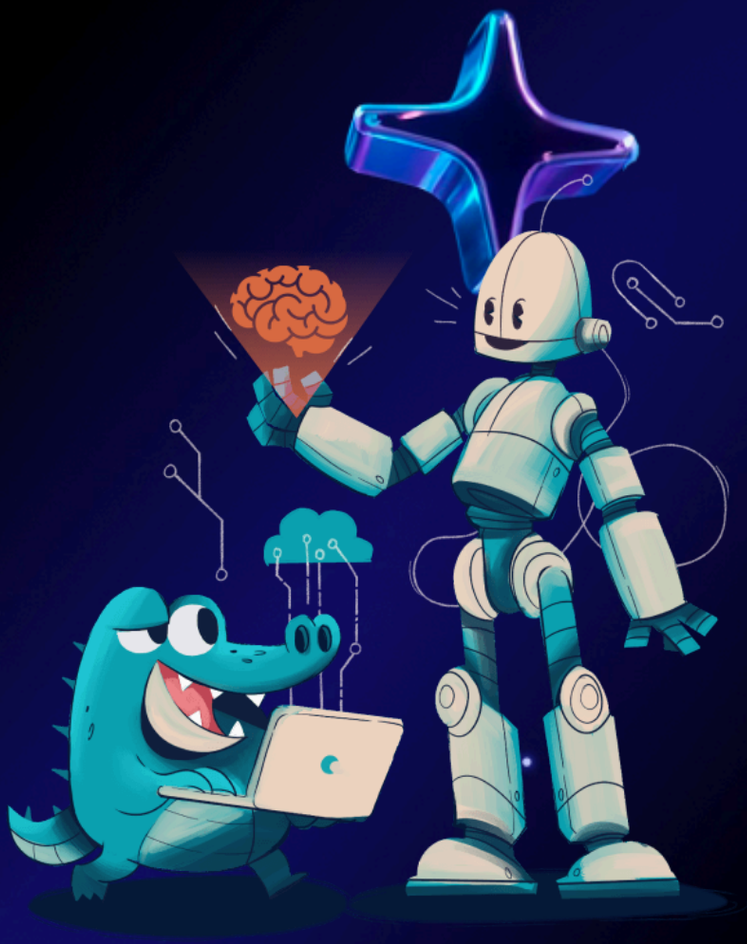
códigofacilito

CLASE 2 · PRODUCT ENGINEER

Cómo construir con IA



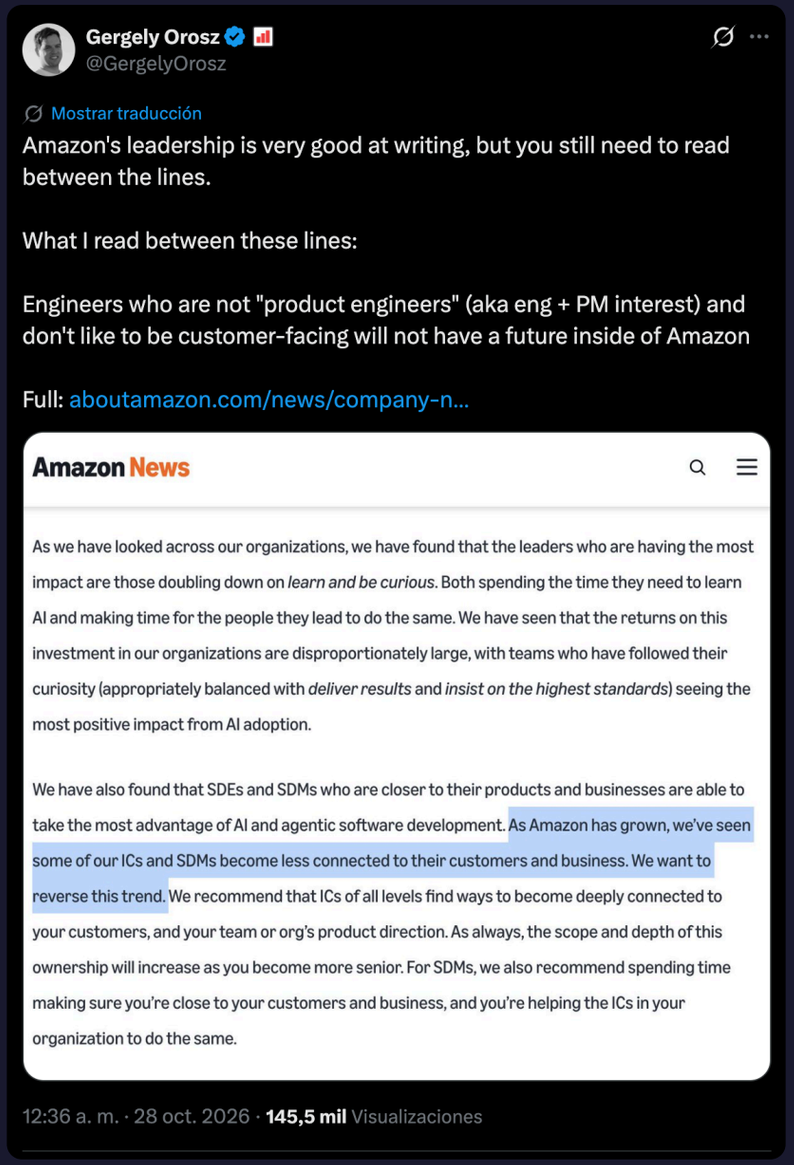
SECCIÓN 1

Introducción

Esta es la semana del Product Engineer

“El renacimiento del ingeniero de software”

Amazon: “Se acabaron los días en que los ingenieros de Amazon no conocen a sus usuarios”



EL FUTURO

El futuro por Felix Rieseberg de Anthropic

Felix Rieseberg, Member of Technical Staff en Anthropic

FELIX RIESEBERG · 1/5

(1) Pick business impact over backlog: In pre-AI-coding 2011, Patrick McKenzie wrote "engineers are hired to create business value, not to program things." Attach yourself, he advised, to the parts of a company that make money. Start there: Understand what your company is actually trying to accomplish (or, at a larger one, what your group or department is). Then find the right altitude — understanding what kind of goals your manager's manager has is often a useful proxy.

FELIX RIESEBERG · 2/5

(2) Pick a radical idea: Come up with a few ideas that would, if successful, dramatically improve the chances that your organization achieves those goals. Be ambitious!

FELIX RIESEBERG · 3/5

(3) Build it over a weekend: I have seen small groups and solo engineers fly past the usual process of writing product requirements documents (PRDs), getting buy-in, and lobbying for time with designers or product managers by building a working prototype.

(4) Focus on one good idea: In my experience, junior people often make the mistake of cramming multiple ideas into a single prototype, hiding three excellent ideas among 20 bad ones. If your prototype involves the creation of a new programming language, a backend, and also somehow mobile apps, you're not going to be successful. The point of creating a working prototype is to communicate an idea so well that leadership quickly understands both the opportunity and that you're the one who can run with it.

FELIX RIESEBERG · 5/5

(5) Lean into your fresh perspective: If you're working on something user-facing, you have a unique amount of empathy for users and thus a much better sense for how the product works or doesn't work for them. My colleagues at Slack and Notion had a hard time internalizing what using Slack or Notion is like as an external user.

Creamos más código que nunca



SECCIÓN 2

¿Qué resultados obtuvimos de la primera tarea?

Qué encontramos

Qué encontramos

- Resultados genéricos

Qué encontramos

- Resultados genéricos
- Plataformas que se ven igual

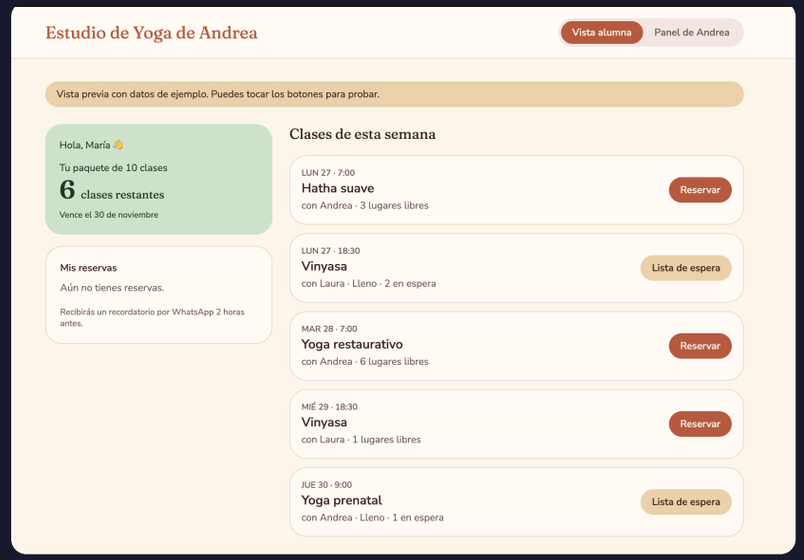
Qué encontramos

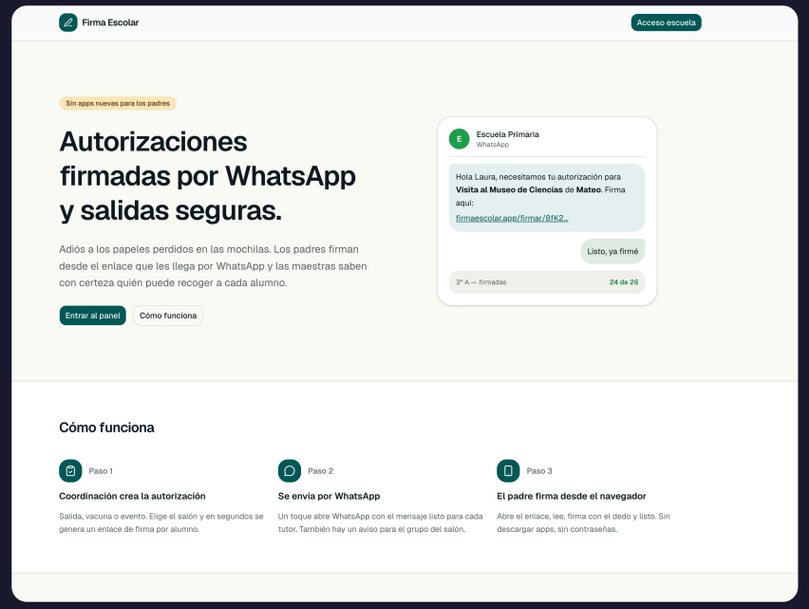
- Resultados genéricos
- Plataformas que se ven igual
- Copys similares

Qué encontramos

- Resultados genéricos
- Plataformas que se ven igual
- Copys similares
- Errores al ingresar a las páginas

Problemas distintos, 2 servicios de IA, resultados casi iguales







Ten personalidad

Ten personalidad

- Muchos prompts estaban hechos con IA

Ten personalidad

- Muchos prompts estaban hechos con IA
- Los LLMs están diseñados para ser promedio

Ten personalidad

- Muchos prompts estaban hechos con IA
- Los LLMs están diseñados para ser promedio
- Nunca vas a ser original si todo lo que produces está hecho con IA

¿Cómo hacerlo mejor?

Ingeniería de prompts →

¿Cómo hacerlo mejor?

Ingeniería de prompts → Ingeniería de contexto

¿Cómo hacerlo mejor?

Ingeniería de prompts → Ingeniería de contexto →

¿Cómo hacerlo mejor?

Ingeniería de prompts → Ingeniería de contexto → **Spec Driven Development**

SECCIÓN 3

Context Engineering

Qué es

Diseñar todo lo que el modelo ve antes de responder, no solo el prompt

Cómo dar contexto

- Prompts e instrucciones (ingeniería de prompts)
- Documentos y datos externos (RAG)
- Estado, herramientas e historial
- Memoria de otras conversaciones
- Formato de salida estructurada

SECCIÓN 4

Spec Driven Development

Qué es SDD

Una fuente única de verdad sobre qué construir y cómo construirlo

Por qué SDD

Entregamos software que funciona, pero no cumple la intención original.

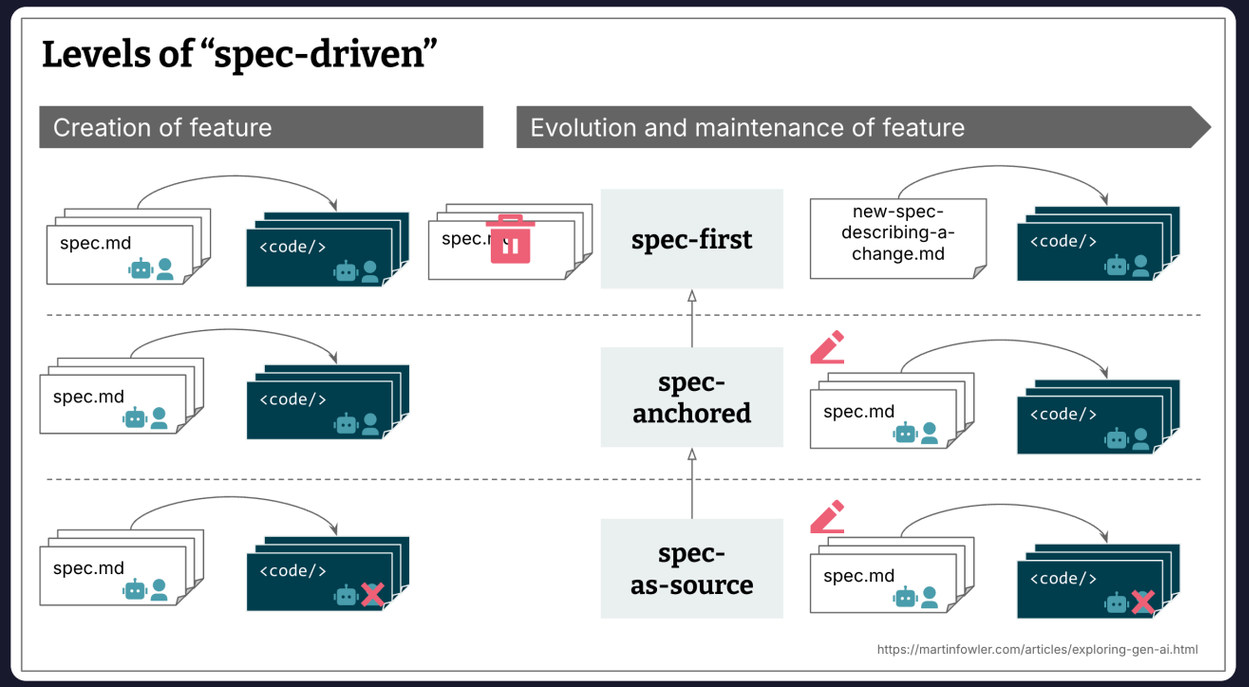
Por qué SDD

Entregamos software que funciona, pero no cumple la intención original.

El problema no es solo la calidad del código.

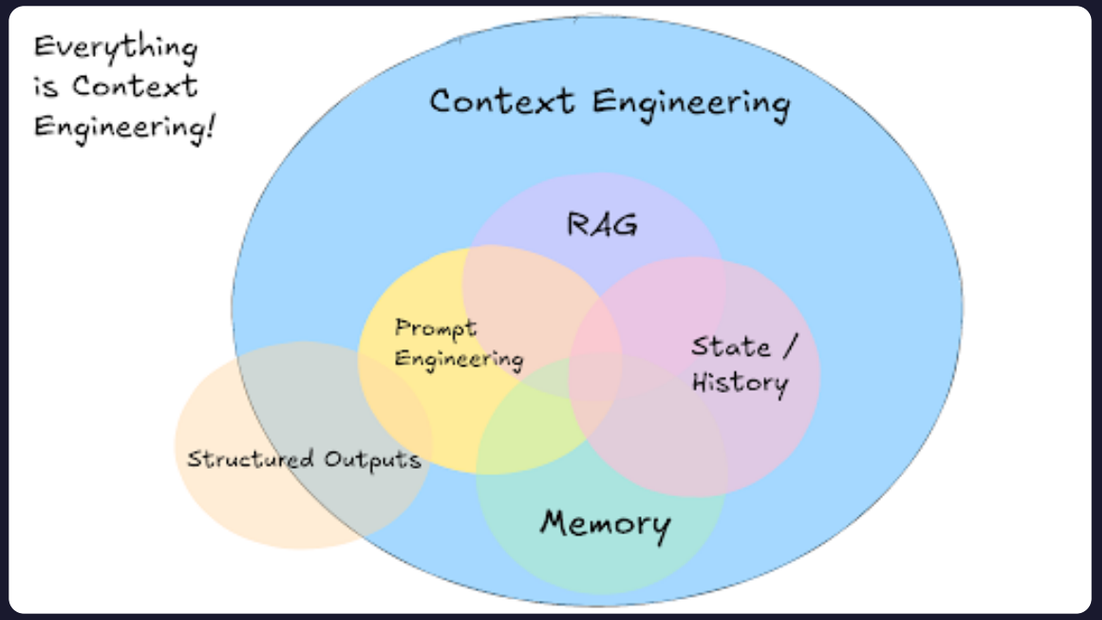
Niveles de SDD

- **Spec-first:** la spec guía la tarea
- **Spec-anchored:** la spec se mantiene para evolucionar el feature
- **Spec-as-source:** el humano solo edita la spec, nunca el código



Spec vs. Contexto

- El contexto vive en todas las sesiones
- La spec vive hasta completar la tarea
- El contexto de una tarea incluye la spec



Cómo funciona

- **Conocimiento persistente:** constitución y steering, aplica a todo

Cómo funciona

- **Conocimiento persistente:** constitución y steering, aplica a todo
- **Spec por tarea:** problema, diseño de la solución y criterios de validación

Ciclo de vida: GitHub Spec Kit



Qué hace una buena spec

Qué hace una buena spec

- Intención, no implementación

Qué hace una buena spec

- Intención, no implementación
- Ni vaga ni sobredetallada

Qué hace una buena spec

- Intención, no implementación
- Ni vaga ni sobredetallada
- Criterios de aceptación verificables

Qué hace una buena spec

- Intención, no implementación
- Ni vaga ni sobredetallada
- Criterios de aceptación verificables
- Un feature o módulo, no todo el sistema

¿Qué tan técnica debe ser?

Mi postura: yo sí hablo de código en la spec

¿Qué tan técnica debe ser?

Mi postura: yo sí hablo de código en la spec

Ahorro tokens, evito duplicados, soy más preciso.

“Sin un plan, solo estás adivinando en código”

Cuándo no funciona el SDD

Fuente: developer.microsoft.com/blog/spec-driven-development-ai-native-engineering

Cuándo no funciona el SDD

- Especificaciones que nadie lee ni revisa

Fuente: developer.microsoft.com/blog/spec-driven-development-ai-native-engineering

Cuándo no funciona el SDD

- Especificaciones que nadie lee ni revisa
- Especificaciones que se quedan obsoletas y no se actualizan

Fuente: developer.microsoft.com/blog/spec-driven-development-ai-native-engineering

Cuándo no funciona el SDD

- Especificaciones que nadie lee ni revisa
- Especificaciones que se quedan obsoletas y no se actualizan
- Proceso pesado para tareas pequeñas

Fuente: developer.microsoft.com/blog/spec-driven-development-ai-native-engineering

Cuándo no funciona el SDD

- Especificaciones que nadie lee ni revisa
- Especificaciones que se quedan obsoletas y no se actualizan
- Proceso pesado para tareas pequeñas

La adopción debe ser a la medida.

Fuente: developer.microsoft.com/blog/spec-driven-development-ai-native-engineering

SECCIÓN 5

Herramientas

Claude Code



Kiro: el agente hecho para SDD

- Genera la spec a partir de tu código
- Requirements → Design → Tasks
- Steering

SECCIÓN 6

Stack técnico

¿El stack importa?

Si eres product engineer importan los resultados, y en los resultados hay $$

¿El stack importa?

Si eres product engineer importan los resultados, y en los resultados hay $$

Mi respuesta: sí importa

Costo computacional

- DigitalOcean droplet básico: **6 USD**
- Vercel Pro: **20 USD** + bandwidth + base de datos
- Render Pro: **20 USD** + bandwidth + base de datos

Costo computacional

- DigitalOcean droplet básico: **6 USD**
- Vercel Pro: **20 USD** + bandwidth + base de datos
- Render Pro: **20 USD** + bandwidth + base de datos

Aquí estás intercambiando dinero por atención, cada producto deberá tomar una decisión.

Costo por token

Lenguajes más eficientes: J, Clojure, Ruby, Python

Language	Avg tokens (Rosetta task)	Type system	Verdict for LLM agents
J	~70	Dynamic	Ultra-compact, niche tradeoff
Clojure	~109	Dynamic	Best for long agent sessions
Ruby	~119	Dynamic	Very token-efficient
Python	~128	Dynamic	Strong balance of cost and readability
Haskell	~130	Static	Lean despite static typing
F#	~136	Static	Efficient typed option
Lisp	~145	Dynamic	Compact, but less mainstream
Scala	~166	Static	Mid-pack, acceptable overhead
JavaScript	~177	Dynamic	Usable, but not especially lean
Go	~182	Static	Moderate token cost
C#	~216	Static	Noticeable token tax
Java	~224	Static	High token tax
C++	~250	Static	Expensive in long contexts
C	~283	Static	Worst for token efficiency

Context awareness

Convención sobre configuración: Go, Rails

Context awareness

Convención sobre configuración: Go, Rails

Rails y los agentes: rubyonrails.org/ai

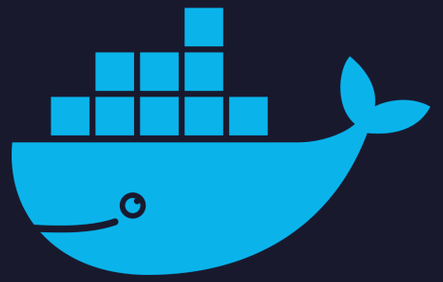
Revisión de código

¿Tu equipo seguirá revisando código?

Mi stack para IA: Rails + Docker



+



TypeScript + Next.js

- La opción por la que optan más IAs
- Mejor infraestructura, Vercel y muchos hostings trabajan sobre este stack

TypeScript + Next.js

- La opción por la que optan más IAs
- Mejor infraestructura, Vercel y muchos hostings trabajan sobre este stack

Mi problema

- No hay una convención establecida
- El agente tiene que averiguar cómo se construyeron las cosas

Por qué Docker

Host vs Container

Por qué Docker

Host vs Container

- A los agentes les gusta instalar, probar, ejecutar cosas

Por qué Docker

Host vs Container

- A los agentes les gusta instalar, probar, ejecutar cosas
- Contenedor: espacio aislado, desechable y reconstruible para tus agentes

Por qué Docker

Host vs Container

- A los agentes les gusta instalar, probar, ejecutar cosas
- Contenedor: espacio aislado, desechable y reconstruible para tus agentes
- ¿Está listo? Lleva esa misma configuración a producción

Por qué Docker

Host vs Container

- A los agentes les gusta instalar, probar, ejecutar cosas
- Contenedor: espacio aislado, desechable y reconstruible para tus agentes
- ¿Está listo? Lleva esa misma configuración a producción
- Docker MCP

Recomendaciones Docker

- Qué es Docker: youtu.be/ikr7wjtz1RA
- Docker para la era de la IA: youtu.be/l7uSR48AIL4

Curso de Docker disponible en el panel del bootcamp

SECCIÓN 7

Práctica

Analicemos las specs

Analicemos los resultados